

Ejercicios Seleccionados

Machine Learning II - Unidad 3

Sesion 1: Arquitectura Transformer en Profundidad

Sesion 2: Acceso Programatico a LLMs

*Selección de los ejercicios mas relevantes para la practica
de Transformers y acceso programatico a LLMs*

SESION 1: Arquitectura Transformer en Profundidad

Ejercicio 1: Calculo Manual de Self-Attention

Duracion: 30 minutos | Tipo: Calculo/Analisis | Dificultad: Intermedia

Objetivo

Comprender el flujo completo del calculo de self-attention. Calcular manualmente las matrices de scores, pesos y salida. Interpretar que representan los pesos de atencion.

Enunciado

Dadas las siguientes matrices Q, K, V para 3 tokens con dimension $d_k = 2$:

$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$,
 $K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0.5 & 0.5 \end{bmatrix}$,
 $V = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$

Paso 1: Calcular los Scores (QK^T)

Multiplica Q por la transpuesta de K. Completa la matriz resultante (3x3).

$K^T = \begin{bmatrix} 1 & 0 & 0.5 \\ 0 & 1 & 0.5 \end{bmatrix}$

$Scores = \begin{bmatrix} _ & _ & _ \\ _ & _ & _ \\ _ & _ & _ \end{bmatrix}$

Pregunta: Cual par de tokens tiene mayor compatibilidad segun los scores sin escalar?

Paso 2: Escalar por raiz de d_k

Divide cada elemento por $\sqrt{d_k} = \sqrt{2} = 1.4142$

Pregunta: Por que escalamos por $\sqrt{d_k}$? Que pasaria con los gradientes si no lo hicieramos?

Paso 3: Aplicar Softmax por filas

Para cada fila, aplica $\text{softmax}(x_i) = \exp(x_i) / \sum(\exp(x_j))$. Calcula las 3 filas mostrando exponenciales, sumas y resultado.

Fila 1: $\exp(\text{scaled}[0][0]) = _$ $\exp(\text{scaled}[0][1]) = _$ $\exp(\text{scaled}[0][2]) = _$
 Suma = $_$ Softmax = $\begin{bmatrix} _ & _ & _ \end{bmatrix}$

Fila 2: ... (repetir)

Fila 3: ... (repetir)

$Attention_Weights = \begin{bmatrix} _ & _ & _ \\ _ & _ & _ \\ _ & _ & _ \end{bmatrix}$

Verifica que cada fila suma 1.0. Por que es importante esta propiedad?

Paso 4: Multiplicar por V

Output = Attention_Weights * V. Completa la matriz resultante (3x2).

Preguntas de Reflexion:

1. El Token 3 (query=[1,1]) tiene compatibilidad alta con todos los keys. Como se refleja en su vector de salida?
2. Si cambiáramos Q[0] de [1,0] a [10,0], como cambiarían los pesos de atencion?
3. Por que es ventajoso tener proyecciones separadas (W_Q, W_K, W_V) en lugar de usar directamente los embeddings?

Ejercicio 2: Visualizacion de Atencion con BertViz

Duracion: 30 minutos | Tipo: Programacion/Exploracion | Dificultad: Intermedia

Objetivo

Configurar y usar BertViz para visualizar patrones de atencion. Identificar que relaciones linguisticas capturan diferentes cabezas. Analizar diferencias entre capas tempranas y profundas.

Parte A: Instalacion y Configuracion (5 min)

```
pip install bertviz transformers torch

from bertviz import head_view, model_view
from transformers import AutoTokenizer, AutoModel
import torch

model_name = "bert-base-multilingual-cased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name, output_attentions=True)
```

Parte B: Visualizacion Basica (10 min)

Usa la funcion visualize_attention() para inspeccionar patrones de atencion en oraciones en espanol.

```
def visualize_attention(sentence, model, tokenizer):
    inputs = tokenizer(sentence, return_tensors="pt")
    tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])
    with torch.no_grad():
        outputs = model(**inputs)
    attentions = outputs.attentions
    head_view(attentions, tokens) # En Jupyter
    return attentions, tokens
```

Parte C: Analisis de Patrones (15 min)

Visualiza la atencion para estas oraciones y responde:

Oracion 1 - Correferencia: "El gato se sento en la alfombra porque estaba cansado"

-> Busca una cabeza donde "estaba"/"cansado" atienda a "gato"

Oracion 2 - Estructura sintactica: "Los estudiantes que aprobaron el examen celebraron con sus amigos"

-> Busca una cabeza donde "celebraron" atienda a "estudiantes"

Oracion 3 - Relaciones a larga distancia: "La empresa que fundaron en Madrid hace diez anos finalmente cerro"

-> Busca una cabeza donde "cerro" atienda a "empresa"

Oracion 4 - Comparacion de idiomas:
ES: "El banco esta cerca del rio" vs EN: "The bank is near the river"
-> Compara patrones para "banco"/"bank" (palabra ambigua)

- Preguntas de Reflexion:**
- 1. Que patrones observas en capas tempranas (0-3) vs profundas (9-11)?
 - 2. Algunas cabezas atienden al token anterior o al [CLS]. Por que son utiles?
 - 3. Cada cabeza se especializa en un tipo de relacion linguistica?

Ejercicio 3: Diseno de un Transformer para un Caso de Uso

Duracion: 25 minutos | Tipo: Diseno/Calculo grupal | Dificultad: Avanzada

Objetivo

Seleccionar hiperparametros de un Transformer de forma justificada. Calcular el numero total de parametros. Comprender los trade-offs entre capacidad del modelo y restricciones de hardware.

Escenario

Disenar un modelo de lenguaje especializado en documentacion tecnica en espanol para una empresa de software. El modelo debe: generar documentacion de APIs, resumir changelogs, responder preguntas sobre documentacion.

- Restricciones:**
- GPU: 1x NVIDIA A100 (80 GB VRAM)
 - Datos: ~5 GB de texto (documentacion tecnica en espanol)
 - Tiempo de entrenamiento: maximo 1 semana
 - Latencia de inferencia: < 100ms por token
 - Longitud maxima de contexto: 4,000 tokens

Parte A: Seleccion de Hiperparametros (10 min)

Completa la tabla justificando cada decision:

Hiperparametro	Valor	Justificacion
-----	-----	-----
Tipo arquitectura (enc/dec)	_____	_____
d_model	_____	_____
num_heads	_____	_____
num_layers	_____	_____
d_ff	_____	_____
seq_length	_____	_____
vocab_size	_____	_____
dropout	_____	_____

Parte B: Calculo de Parametros (10 min)

Calcula el total de parametros usando estas formulas:

Embedding: vocab_size * d_model
Atencion: 4 * d_model^2 + 4 * d_model (por capa)

```

FFN:      d_model*d_ff + d_ff + d_ff*d_model + d_model  (por capa)
LayerNorm: 2 * (2 * d_model)  (por capa)
Salida:    d_model * vocab_size  (o compartido con embedding)

Total = Embedding + num_layers*(Atencion+FFN+LN) + Salida
Memoria FP32 = Total * 4 bytes
Memoria entrenamiento ~ Memoria * 4  (modelo+gradientes+optimizador)

```

Parte C: Comparacion con Modelos Conocidos (5 min)

Modelo	Params	d_model	Capas	Cabezas	d_ff	Vocab
-----	-----	-----	-----	-----	-----	-----
BERT-base	110M	768	12	12	3072	30,522
GPT-2 Small	124M	768	12	12	3072	50,257
GPT-2 Medium	355M	1024	24	16	4096	50,257
LLaMA-7B	7B	4096	32	32	11,008	32,000

Preguntas: 1) Tu modelo es comparable a cual referencia? 2) Con 5GB de datos, es adecuado el tamaño? (regla: 10-20 tokens/parametro) 3) Con el doble de VRAM, que cambiarías primero? 4) Usarías weight tying (compartir embedding y salida)?

SESION 2: Acceso Programatico a LLMs

Nota: En todos los ejercicios puedes usar OpenRouter (gratis) como alternativa a APIs de pago. Configura: client = OpenAI(base_url='https://openrouter.ai/api/v1', api_key=os.getenv('OPENROUTER_API_KEY'))

Ejercicio 4: Chatbot con Memoria

Duracion: 35 minutos | Tipo: Programacion | Dificultad: Intermedia

Objetivo

Implementar gestion de historial de conversacion. Entender como funciona la ventana de contexto. Aplicar limites de tokens mediante recorte de historial. Usar el rol system para definir personalidad.

Paso 1: Estructura base (10 min)

Implementa un chatbot con historial de mensajes. Completa las funciones:

```
import os
from openai import OpenAI
from dotenv import load_dotenv

load_dotenv()
client = OpenAI() # o OpenRouter
MODEL = "gpt-4o-mini"

# TODO: Define system prompt para tutor de Python amigable
SYSTEM_PROMPT = ____
MAX_MESSAGES = 10

def create_initial_messages():
    """Lista inicial con el system prompt."""
    return ____

def trim_history(messages):
    """Recorta historial si > MAX_MESSAGES.
    Mantiene siempre el system prompt (messages[0])."""
    if len(messages) - 1 > MAX_MESSAGES:
        ____
    return messages

def get_response(messages):
    """Llama a la API con el historial completo."""
    return ____

def chat():
    messages = create_initial_messages()
    while True:
        user_input = input("Tu: ").strip()
        if user_input.lower() == "salir": break
```

```
# TODO: Anadir msg usuario, recortar, llamar API,
#         extraer respuesta, anadir al historial
```

Paso 2: Prueba de memoria (10 min)

Prueba esta secuencia:

1. "Que son las variables en Python?"
2. "Dame un ejemplo de lo anterior"
3. "Ahora muestrame como usar listas"
4. "Cual es la diferencia entre lo primero que me explicaste y esto?"

Verifica que recuerda el contexto en cada pregunta.

Paso 3: Probar limite de historial (10 min)

Cambia MAX_MESSAGES=4 y manten una conversacion larga. Observa:

- En que momento el chatbot 'olvida' los primeros mensajes?
- Como afecta a la coherencia?
- Como cambia el consumo de tokens cuando se recorta?

Bonus: Resumen de historial

En lugar de eliminar mensajes antiguos, genera un resumen con la API y anadelo como contexto adicional al system prompt.

Ejercicio 5: Extraccion Estructurada con JSON

Duracion: 25 minutos | Tipo: Programacion | Dificultad: Avanzada

Objetivo

Disenar prompts que produzcan JSON valido. Implementar validacion y manejo de errores. Construir logica de reintentos cuando la respuesta no cumple el formato.

System Prompt de extraccion:

```
SYSTEM_PROMPT = """Eres un sistema de extraccion de informacion.
Extrae datos estructurados de textos y devuelve UNICAMENTE JSON valido.
Reglas:
- Responde SOLO con JSON, sin texto adicional
- Campo no encontrado -> "No especificado"
- Valores numericos como numeros, no strings
- Fechas en formato YYYY-MM-DD"""
```

Textos de prueba:

Texto 1 - Oferta de empleo: Desarrollador Senior Python en Madrid, 45.000-55.000 EUR, teletrabajo 3 dias, 5 anos experiencia, Django+PostgreSQL.

-> Extraer: puesto, empresa, ubicacion, salario_min/max, modalidad, requisitos, beneficios

Texto 2 - Resena de producto: Portatil UltraBook X15, pantalla 15", 10h bateria, teclado incomodo, 1.299 EUR, 7/10, comprado en Amazon.

-> Extraer: producto, puntuacion, precio, aspectos_positivos/negativos, recomendacion

Texto 3 - Noticia: NovaTech, Serie B, 30M EUR, Sequoia Capital, fundada 2021, expansion a Latinoamerica, 50 ingenieros, sector legal.

-> Extraer: empresa, tipo_evento, monto, inversores, fundadores, sector, planes

Implementacion con reintentos:

```
def extract_json(text, schema_description, max_retries=3):
    for attempt in range(max_retries):
        try:
            response = client.chat.completions.create(
                model=MODEL,
                messages=[
                    {"role": "system", "content": SYSTEM_PROMPT},
                    {"role": "user", "content": f"Extrae...{schema}...{text}"}
                ],
                temperature=0
            )
            content = response.choices[0].message.content.strip()
            # Limpiar bloques markdown si los hay
            if content.startswith("```"):
                content = content.split("\n",1)[1].rsplit("```",1)[0]
            return json.loads(content)
        except json.JSONDecodeError as e:
            print(f"Intento {attempt+1}: JSON invalido: {e}")
    return None
```

Analisis:

1. En cuantos intentos genero JSON valido para cada texto?
2. Hubo campos con 'No especificado'? Era correcto?
3. Los valores numericos fueron numeros o strings?
4. Que pasaria si el texto estuviera en otro idioma?

Ejercicio 6: Introduccion a LangChain

Duracion: 30 minutos | Tipo: Programacion | Dificultad: Intermedia

Objetivo

Instalar y configurar LangChain con OpenAI. Crear cadenas (chains) usando el operador pipe (|). Comparar complejidad del codigo nativo vs LangChain.

Paso 1: Instalacion (5 min)

```
pip install langchain langchain-openai

import langchain
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```


Paso 2: Primera chain (10 min)

Replica la extraccion del ejercicio anterior usando LangChain:

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Modelo
model = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# 2. Template del prompt
prompt = ChatPromptTemplate.from_messages([
    ("system", ""Eres un sistema de extraccion...""),
    ("user", ""Extrae del texto con esquema: {schema}
    Texto: {text}""")
])

# 3. Parser de salida
output_parser = StrOutputParser()

# 4. Chain con operador pipe
chain = prompt | model | output_parser

# 5. Invocar
result = chain.invoke({"text": texto, "schema": esquema})
```

Paso 3: Procesar los tres textos (10 min)

Usa la misma chain para los 3 textos del ejercicio anterior y parsea los resultados con `json.loads()`.

Paso 4: Comparacion (5 min)

Aspecto	API Nativa	LangChain
-----	-----	-----
Lineas de codigo		
Facilidad de lectura		
Gestion de reintentos		
Cambiar de modelo		
Curva de aprendizaje		

Preguntas de Reflexion:

1. Cuantas lineas de codigo te ahorraste con LangChain?
2. Que beneficios ves en el patron `prompt | model | parser`?
3. En que situaciones NO usarias LangChain y preferirias la API nativa?
4. Como cambiarias la chain para usar Claude en vez de OpenAI?